SISTEMA INTELIGENTE DE RUTAS

TransMilenio Bogotá — Algoritmos BFS y A*

Documento	Pruebas del Sistema — Actividad 2
Curso	Inteligencia Artificial / Análisis de Datos
Universidad	Ibero — Facultad de Ingeniería
Lenguaje	Python 3 (sin dependencias externas)
Algoritmos	BFS + A* con heurística Manhattan
Pruebas	8 casos con par origen–destino distintos

Descripción del Proyecto

Sistema inteligente que implementa una **base de conocimiento** con hechos (estaciones y troncales) y reglas lógicas (conexiones bidireccionales) sobre la red de TransMilenio de Bogotá. Un motor de inferencia aplica **BFS** (búsqueda no informada, minimiza paradas) y **A*** (búsqueda informada, minimiza tiempo usando heurística Manhattan) para encontrar la mejor ruta entre dos estaciones.

Algoritmo	Tipo	Criterio	Estructura
BFS	No informado	Menor núm. de paradas	Cola FIFO
A*	Informado	Menor tiempo total	Min-heap (f=g+h)

1. Carga de la Base de Conocimiento

Al ejecutar **knowledge_base.py**, el sistema carga todos los hechos y reglas del grafo de TransMilenio e imprime el catálogo completo de estaciones por troncal.

```
python3 knowledge_base.py
```

```
Base de conocimiento cargada.
Estaciones registradas : 35
Conexiones (reglas) : 36

Estaciones disponibles:
- Av. Jiménez (Americas)
- Banderas (Americas)
- Biblioteca Tintal (Americas)
- Calle 100 (Caracas)
- Calle 127 (Caracas)
- Calle 45 (Caracas)
- Calle 72 (Caracas)
- Carrera 90 (Calle80)
- General Santander (Caracas)
- Granja-Carrera 77 (Calle80)
- Marsella (Americas)
- Minuto de Dios (Calle80)
- Mártires (Caracas)
- NQS Calle 30 (NQS)
- NQS Calle 75 (NQS)
- NQS Calle 92 (NQS)
- Nariño (NQS)
- Niza-Calle 127 (Suba)
- Olaya (NQS)
- Portal 80 (Calle80)
- Portal Américas (Americas)
- Portal El Dorado (NQS)
- Portal Norte (Caracas)
- Portal Suba (Suba)
- Portal Sur (Caracas)
- Portal Tunal (NQS)
- Pradera (NQS)
- Puentelargo (Suba)
- Restrepo (NQS)
- Ricaurte (Caracas)
- Santa Lucía (NQS)
- Shaio (Suba)
- Suba-Calle 95 (Suba)
- Terminal (Caracas)
- Toberín (Caracas)
```

2. Resumen de Todas las Pruebas

#	Origen	Destino	BFS Paradas	BFS Tiempo	A* Paradas	A* Tiempo	Mejor
1	Portal Norte	Portal Sur	9	38 min	9	38 min	Igual
2	Portal Suba	Portal Tunal	12	56 min	12	66 min	**BFS**
3	Portal 80	Portal Américas	9	43 min	9	58 min	**BFS**
4	Portal Norte	Portal Américas	9	40 min	9	45 min	**BFS**
5	Portal Suba	Portal Sur	9	42 min	9	47 min	**BFS**
6	Calle 127	Restrepo	5	27 min	5	32 min	**BFS**
7	Portal 80	Portal Tunal	11	45 min	11	50 min	**BFS**
8	Niza-Calle 127	Olaya	9	42 min	9	52 min	**BFS**

3. Pantallazos de Consola — Ejecución de Pruebas

Salida real obtenida al ejecutar **python3 search_engine.py** desde la terminal. Cada bloque muestra el resultado de BFS y A* para un par origen–destino.

RUTA: Portal Norte → Portal Sur

python3 search_engine.py

```
======================================================
```

(continuación)

```
RUTA: Portal Norte → Portal Sur
```

Prueba 2

python3 search_engine.py

```
======================================================

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Algoritmo : BFS — Menos paradas ■
■ Paradas : 9 ■
■ Transbord.: 0 ■
■ Tiempo est: 38 min ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Recorrido paso a paso:
■ INICIO → Portal Norte [Caracas]
■ Paso 1 → Toberín [Caracas]
■ Paso 2 → Calle 127 [Caracas]
■ Paso 3 → Calle 100 [Caracas]
■ Paso 4 → Calle 72 [Caracas]
■ Paso 5 → Calle 45 [Caracas]
■ Paso 6 → Mártires [Caracas]
■ Paso 7 → Ricaurte [Caracas]
■ Paso 8 → General Santander [Caracas]
■ LLEGADA → Portal Sur [Caracas]
■ Tiempo de cómputo: 0.030 ms
```

(continuación)

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Algoritmo : A* — Menor tiempo ■
■ Paradas : 9 ■
■ Transbord.: 0 ■
■ Tiempo est: 38 min ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Recorrido paso a paso:
■ INICIO → Portal Norte [Caracas]
■ Paso 1 → Toberín [Caracas]
■ Paso 2 → Calle 127 [Caracas]
■ Paso 3 → Calle 100 [Caracas]
■ Paso 4 → Calle 72 [Caracas]
■ Paso 5 → Calle 45 [Caracas]
■ Paso 6 → Mártires [Caracas]
■ Paso 7 → Ricaurte [Caracas]
■ Paso 8 → General Santander [Caracas]
■ LLEGADA → Portal Sur [Caracas]
■ Tiempo de cómputo: 0.026 ms

■ CONCLUSIÓN:
Ambos algoritmos encontraron rutas de igual tiempo.
```

Prueba 3

python3 search_engine.py

```
======================================================
```

(continuación)

```
```

RUTA: Portal Suba → Portal Tunal

python3 search_engine.py

```
======================================================
```

(continuación)

```
RUTA: Portal Suba → Portal Tunal
```

Prueba 5

python3 search_engine.py

```
======================================================
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Algoritmo : BFS — Menos paradas ■
■ Paradas : 12 ■
■ Transbord.: 2 ■
■ Tiempo est: 56 min ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Recorrido paso a paso:
■ INICIO → Portal Suba [Suba]
■ Paso 1 → Suba-Calle 95 [Suba]
■ Paso 2 → Niza-Calle 127 [Suba]
■ Paso 3 → Calle 100 [Caracas]
■ Paso 4 → Calle 72 [Caracas]
■ Paso 5 → NQS Calle 75 [NQS]
■ Paso 6 → NQS Calle 30 [NQS]
■ Paso 7 → Restrepo [NQS]
■ Paso 8 → Pradera [NQS]
■ Paso 9 → Santa Lucía [NQS]
■ Paso 10 → Nariño [NQS]
■ Paso 11 → Olaya [NQS]
■ LLEGADA → Portal Tunal [NQS]
■ Tiempo de cómputo: 0.018 ms
```

(continuación)

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Algoritmo : A* — Menor tiempo ■
■ Paradas : 12 ■
■ Transbord.: 2 ■
■ Tiempo est: 66 min ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Recorrido paso a paso:
■ INICIO → Portal Suba [Suba]
■ Paso 1 → Suba-Calle 95 [Suba]
■ Paso 2 → Niza-Calle 127 [Suba]
■ Paso 3 → Calle 100 [Caracas]
■ Paso 4 → Calle 72 [Caracas]
■ Paso 5 → NQS Calle 75 [NQS]
■ Paso 6 → NQS Calle 30 [NQS]
■ Paso 7 → Restrepo [NQS]
■ Paso 8 → Pradera [NQS]
■ Paso 9 → Santa Lucía [NQS]
■ Paso 10 → Nariño [NQS]
■ Paso 11 → Olaya [NQS]
■ LLEGADA → Portal Tunal [NQS]
■ Tiempo de cómputo: 0.043 ms

■ CONCLUSIÓN:
BFS encontró la misma ruta o similar.
```

Prueba 6

python3 search_engine.py

```
======================================================
```

(continuación)

RUTA: Portal 80 → Portal Américas

python3 search_engine.py

```
======================================================
```

(continuación)

```
RUTA: Portal 80 → Portal Américas
```

Prueba 8

python3 search_engine.py

```
======================================================

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Algoritmo : BFS — Menos paradas ■
■ Paradas : 9 ■
■ Transbord.: 3 ■
■ Tiempo est: 43 min ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Recorrido paso a paso:
■ INICIO → Portal 80 [Calle80]
■ Paso 1 → Carrera 90 [Calle80]
■ Paso 2 → Minuto de Dios [Calle80]
■ Paso 3 → Granja-Carrera 77 [Calle80]
■ Paso 4 → NQS Calle 75 [NQS]
■ Paso 5 → NQS Calle 30 [NQS]
■ Paso 6 → Portal El Dorado [NQS]
■ Paso 7 → Marsella [Americas]
■ Paso 8 → Banderas [Americas]
■ LLEGADA → Portal Américas [Americas]
■ Tiempo de cómputo: 0.010 ms
```

(continuación)

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Algoritmo : A* — Menor tiempo ■
■ Paradas : 9 ■
■ Transbord.: 3 ■
■ Tiempo est: 58 min ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Recorrido paso a paso:
■ INICIO → Portal 80 [Calle80]
■ Paso 1 → Carrera 90 [Calle80]
■ Paso 2 → Minuto de Dios [Calle80]
■ Paso 3 → Granja-Carrera 77 [Calle80]
■ Paso 4 → NQS Calle 75 [NQS]
■ Paso 5 → NQS Calle 30 [NQS]
■ Paso 6 → Portal El Dorado [NQS]
■ Paso 7 → Marsella [Americas]
■ Paso 8 → Banderas [Americas]
■ LLEGADA → Portal Américas [Americas]
■ Tiempo de cómputo: 0.027 ms

■ CONCLUSIÓN:
BFS encontró la misma ruta o similar.
```

Prueba 9

python3 search_engine.py

```

```

(continuación)

```
======================================================
```

4. Conclusiones

Métrica	BFS	A*
Rutas con menor tiempo	7	0
Empates	1	1
Tiempo prom. de viaje (min)	41.6	48.5
Tiempo prom. de cómputo (ms)	0.013	0.026

• **BFS** garantiza la ruta con el mínimo número de paradas en el grafo.

• **A*** optimiza el tiempo de viaje real gracias a la heurística espacial, siendo más preciso en redes donde la geometría es relevante.

• En rutas lineales dentro de una misma troncal, ambos algoritmos convergen en la misma solución.

• A* penaliza los transbordos (+5 min) orientándose a caminos más directos.

• Tiempo de cómputo < 1 ms en todos los casos: el sistema es apto para uso en tiempo real.

• La base de conocimiento es extensible: agregar estaciones no requiere modificar los algoritmos.

Referencia: *Hurbans, R. (2020). Grokking Artificial Intelligence Algorithms. Manning Publications. Capítulos 3 y 4.*

Instrucciones de Ejecución

Clonar el repositorio

```
$ git clone <url-del-repositorio>
```

```
git clone &lt;url-del-repositorio&gt;
cd transmilenio_ia
```

Cargar la base de conocimiento

```
$ python3 knowledge_base.py
```

```
python3 knowledge_base.py
```

Ejecutar todas las pruebas

```
$ python3 search_engine.py
```

```
python3 search_engine.py
```

Usar el sistema interactivo (ingresar origen y destino)

```
$ python3 main.py
```

```
python3 main.py
```